Decomposition and Data Modeling

Working reference and drill plan for turning a problem into a program (C or Rust)

> **The gap this guide targets:** you know the syntax but cannot turn "what I want" into types, functions and a program structure. That is a separate skill from syntax. It has three parts, trained in this order: **model** the data, **decompose** the work, **recognize** which known structure or pattern fits.

1. Why composition feels abstract

Syntax has right answers, so tutorials and courses can teach and grade it. Composition is a series of design decisions with trade-offs, and the decisions are invisible in finished code. In most courses the instructor has already decomposed the problem for you ("write a function that does X"), so the hardest step is never practiced. You build the skill only by making these decisions repeatedly and seeing their consequences.

Core model: a program is **data plus transformations**. The data shape comes from the problem. The operations come from what that data must do. Once both are chosen, the code is mostly mechanical. Bad programs usually come from a wrong data shape, not from wrong loops.

Skill	Question it answers	Output	How it is trained
Model	What things exist, what do I need to know about each, how many, how do I find them?	Types, collections, invariants	Worksheet on every project; modeling drills (section 11)
Decompose	What happens, in what order, and what are the smallest pieces?	Function signatures, call structure	Top-down outline, stub-first builds
Recognize	Have I seen this shape before? Which structure or pattern fits?	A choice of structure or algorithm	Learning the standard structures; solving problems by pattern

2. The design worksheet (use on every project)

Answer in your own words on paper **before** writing logic. Every skipped row is a bug or rewrite later. Copy this page for each project.

Prompt	Your answer
1. Input and output. Given ____, produce ____. (If you cannot say it plainly, stop.)	
2. Nouns. What things exist? For each: what do I need to know about it?	
3. Choices. Which values are one of a fixed set? (becomes an enum)	
4. Collections. How many of each? Looked up by key, processed in order, or by priority?	
5. Verbs. What actions exist? For each: what goes in, what comes out? (signatures only)	
6. Invariants. What must always be true about the data? (e.g. count never exceeds capacity)	
7. Edge cases. Empty, full, one item, duplicates, bad input, zero, huge?	
8. Smallest slice. The simplest input-to-output path I can run first?	

Then write the types and function signatures with empty bodies. That is the design. Function bodies are the part you already know how to write.

3. Data modeling in depth

3.1 Find the nouns, keep the ones that carry data

Write the problem in plain sentences. Underline nouns. Keep a noun as a type only if there is something to know about it. Merge synonyms. A "battery test" has a rated CCA, a measured CCA, a voltage and a verdict, so it earns a struct. "The technician" earns nothing unless the program stores something about them.

3.2 Decide fields by what the verbs need

- Ask for each field: which function reads it? If none, delete it.
- Store **raw facts**, derive the rest. Do not store an average next to the values it comes from; the two will drift apart.
- Fields that change together belong in one type. Fields that change independently may belong in separate types.
- Use a name from the problem domain (`BatteryTest`), not from the computer (`Data2`).

3.3 Choices and variants

When a value is "one of" a fixed set, use an enum. When each option carries different data, you need a tagged variant. Rust makes this explicit; C needs a tag plus a union.

Problem concept	C	Rust
Thing with fields	struct	struct
One of a fixed set	enum	enum
One of several shapes, each with its own data	struct with a tag field + union	enum with data: Shape::Circle(f64)
Might not exist	NULL or sentinel value (easy to forget to check)	Option<T> (compiler forces the check)
Might fail	return code + out-parameter	Result<T, E>
Growable list	pointer + len + cap, manual realloc	Vec<T>

3.4 Make bad states impossible to represent

The strongest modeling habit: shape the type so that invalid combinations cannot be written. Compare:

```
/* Weak: two fields that can disagree */
struct Test { int tested; int result; };      /* result is meaningless when tested == 0 */

/* Weak: parallel arrays, indexes can drift apart */
int    ids[100];  double volts[100];  int results[100];

/* Better: one record per thing, one enum covering every state */
typedef enum { UNTESTED, GOOD, CHARGE, REPLACE } Verdict;
typedef struct { int id; double volts; Verdict verdict; } Test;
Test tests[100];
```

3.5 Invariants

An invariant is a fact that must hold before and after every operation. Write them as comments above the type, then check that each function preserves them. This is how you reason about correctness instead of hoping.

```
/* Invariants: count <= MAX_TESTS.  tests[0..count-1] are valid; the rest are unused. */
typedef struct { BatteryTest tests[MAX_TESTS]; size_t count; } TestLog;
```

3.6 Pick the collection from the access pattern

Ask how the program will use the collection, not what you used last time:

Phrase in the problem	Structure
do something with every item, in order; get item N	array / Vec
find by name, ID or key; count occurrences; have I seen this?	hash map / hash set
most recent first; undo; matching nested things	stack
first come first served; keep the last N readings	queue / ring buffer
always need the smallest or largest next	heap (priority queue)
keep sorted, insert often, range queries	balanced tree or sorted array
things connected to other things	graph (adjacency list)
a fixed set of named states with allowed transitions	enum + state machine
bits on or off (flags, pin states)	bit flags / bitset

3.7 Ownership: who creates, changes and frees the data

Every piece of data needs an answer to these. In C you answer by convention and discipline; in Rust the compiler makes you answer up front. It is the same question either way.

- Function only reads: pass a `const T *` (C) or `&T` (Rust).
- Function changes it: pass `T *` or `&mut T`.
- Small plain value: pass by value.
- Heap memory: decide which function allocates and which frees, and write it in a comment.

4. Decomposition in depth

4.1 Top-down outline

Start with one function that states the whole job in 3 to 7 plain steps, as comments. Each step becomes a function. Repeat on any step that is still not trivial.

```
int main(void) {
    /* 1. create an empty log
       2. for each battery: read values, classify, add to log
       3. report: count per verdict, average voltage */
}
```

4.2 Stub first, then fill in

Write every function as a signature that returns a dummy value. Get the whole program compiling and running with fake results. Then replace stubs one at a time, testing each. You always have a working program, and the structure is settled before any detail can distract you.

4.3 Run an example by hand

Take one concrete input and work it out on paper. The steps you took are the algorithm. If you cannot do it by hand, you do not yet understand the problem; no amount of code fixes that.

4.4 One job per function

- Naming test: describe the function with one verb phrase and no "and". If you need "and", split it.
- Keep one level of abstraction per function. A function that both loops over batteries and does byte-level parsing mixes levels.

- Separate **pure** functions (inputs to outputs, no I/O) from **effectful** ones (printing, reading, hardware). Pure functions are trivial to test; keep the effectful layer thin.
- Duplicated logic means a function or type is missing.

4.5 When you are stuck

Symptom	Move
Blank file, no idea where to start	Write the Input/Output sentence. Then do one example by hand.
Problem feels too big	Solve a smaller version: one item, fixed input, no error handling. Then widen.
Do not know what the types are	Re-read the problem; list nouns; ask what each verb needs to read.
Do not know which function comes first	Write the top-level outline as comments; the order falls out.
Code works but is hard to change	The data shape is probably wrong. Revisit nouns and collection choice.

5. Worked example 1: battery test log (full loop)

Problem: a technician tests batteries and wants a verdict for each plus a summary. Worksheet filled in:

Prompt	Answer
1. Input/Output	Given rated CCA, measured CCA and voltage per battery, produce a verdict per battery and a summary: count per verdict and average voltage.
2. Nouns	BatteryTest: id, rated_cca, measured_cca, voltage, result. The log of tests.
3. Choices	Verdict is one of good / charge / replace: enum.
4. Collections	Many tests, processed in order, never looked up by key, at most 100: array plus a count.
5. Verbs	classify(rated, measured, voltage) -> verdict. log_add(log, test) -> ok or full. log_avg_voltage(log) -> double. log_count_result(log, verdict) -> count.
6. Invariants	count <= MAX_TESTS. Every stored test has a result set.
7. Edge cases	Empty log (average divides by zero). Full log. rated_cca of 0. Voltage out of range.
8. Smallest slice	One test through classify, printed.

Types and signatures first (the design), then bodies:

```
#include <stdio.h>
#include <stddef.h>

#define MAX_TESTS 100

typedef enum { RESULT_GOOD, RESULT_CHARGE, RESULT_REPLACE } Result;

typedef struct {
    int    id;
    int    rated_cca;      /* from the label */
    int    measured_cca;   /* from the tester */
    double voltage;
    Result result;
} BatteryTest;

/* Invariants: count <= MAX_TESTS; tests[0..count-1] are valid. */
typedef struct {
    BatteryTest tests[MAX_TESTS];
    size_t      count;
} TestLog;

Result classify(int rated_cca, int measured_cca, double voltage) {
```

```
    if (voltage < 12.4) return RESULT_CHARGE;                  /* illustrative thresholds: */
    if (measured_cca * 100 < rated_cca * 70) return RESULT_REPLACE;   /* use your real rules */
    return RESULT_GOOD;
}

int log_add(TestLog *log, BatteryTest t) {           /* returns 0 on success, -1 if full */
    if (log->count >= MAX_TESTS) return -1;
    log->tests[log->count++] = t;
    return 0;
}

double log_avg_voltage(const TestLog *log) {
    if (log->count == 0) return 0.0;                 /* edge case handled explicitly */
    double sum = 0.0;
    for (size_t i = 0; i < log->count; i++) sum += log->tests[i].voltage;
    return sum / (double)log->count;
}

size_t log_count_result(const TestLog *log, Result r) {
    size_t n = 0;
    for (size_t i = 0; i < log->count; i++)
        if (log->tests[i].result == r) n++;
    return n;
}

int main(void) {
    TestLog log = {0};
    BatteryTest t = { .id = 1, .rated_cca = 600, .measured_cca = 540, .voltage = 12.6 };
    t.result = classify(t.rated_cca, t.measured_cca, t.voltage);
    log_add(&log, t);
    printf("avg V: %.2f, good: %zu\n", log_avg_voltage(&log), log_count_result(&log, RESULT_GOOD));
    return 0;
}
```

Execution status: compiled with `gcc -Wall -Wextra` with no warnings and run; output was `avg V: 12.60, good: 1`. The verdict thresholds are placeholders for illustration, not real testing rules.

What to notice

- The types (about 15 lines) settled the whole program. The functions followed from them.
- `classify` is pure: no printing, no reading. You can test it by calling it with numbers.
- The collection choice came from "in order, never looked up by key". If you later need "find test by ID", that answer changes and so does the structure.
- The empty-log edge case was caught at the worksheet stage, not found by a crash.

Self-prompt extension (do it yourself, then bring it to chat)

- Add "find the worst battery by measured / rated ratio". Which nouns, fields or verbs change?
- Add "look up a test by id". Does the collection choice still hold? What would you change?
- What happens at 101 tests? Should log_add reject, overwrite the oldest, or grow? What does that choice imply for the type?

6. Worked example 2: recognizing a pattern (bracket matcher)

Problem: given a string, decide whether its brackets `()` `[]` `{}` are balanced.

- **Input/Output:** given a string, produce yes or no.
- **Do an example by hand:** `a(b[c]{d})`. Each closer must match the most recent opener that is still unmatched. So: remember openers; when a closer arrives, compare it to the latest one and forget that one.
- **Recognize:** "most recent unmatched thing comes back first" is last-in-first-out. That is a **stack**.
- **Edge cases:** closer with an empty stack; openers left over at the end; stack overflow; empty string.

```
int balanced(const char *s) {
    char stack[256];
    size_t top = 0;
    for (; *s; s++) {
        char c = *s;
        if (c == '(' || c == '[' || c == '{') {
            if (top == sizeof stack) return 0;            /* overflow */
            stack[top++] = c;
        } else if (c == ')' || c == ']' || c == '}') {
            if (top == 0) return 0;                       /* closer with nothing open */
            char open = stack[--top];
            if ((c == ')' && open != '(') || (c == ']' && open != '[') ||
                (c == '}' && open != '{')) return 0;      /* wrong kind */
        }
    }
    return top == 0;                                      /* leftovers mean unbalanced */
}
```

Execution status: compiled cleanly and checked on four inputs: balanced mixed brackets returned 1, `(]` returned 0, `((` returned 0, and the empty string returned 1. It is not exhaustively tested.

The lesson: the hard step was not the code. It was noticing the problem has stack shape. That recognition comes from knowing the standard structures well enough that their shapes are familiar (section 7).

7. Core data structures (learn in this order)

For each: know what it is, when to reach for it, the cost of its main operations, and how to build one yourself. Building them is the most direct way to practice modeling, because you must design the type and the operations with a known right answer to check against.

Structure	Main operations (cost)	Build it yourself (C)	Common pitfalls
Dynamic array	index O(1); push amortized O(1); insert/delete in middle O(n)	struct { T *data; size_t len, cap; }; double capacity with realloc when full	realloc can move memory and invalidate pointers; check for NULL
Stack	push, pop, peek O(1)	array + top index (or dynamic array)	overflow and underflow checks
Ring buffer / queue	enqueue, dequeue O(1), fixed size	array + head index + count; wrap with modulo	decide in advance: reject or overwrite when full
Linked list	insert/remove at a known node O(1); find O(n)	node { T value; Node *next; }; keep head pointer	poor cache behavior; free every node; rarely the best choice, but teaches pointers
Hash map / set	lookup, insert, delete average O(1)	array of buckets, hash function, collision handling (chaining), resize at a load factor	weak hash, resize bugs, who owns the key memory
Binary heap	push, pop O(log n); peek O(1)	array as a complete binary tree: children of i at 2i+1 and 2i+2; sift up and sift down	off-by-one in index math; restore the heap invariant after every change
Binary search tree	search, insert average O(log n)	node { key; Node *left, *right; }	unbalanced worst case O(n); deletion is tricky
Graph	depends on the algorithm; BFS/DFS O(V+E)	adjacency list: array of lists, one per vertex; visited array	forgetting to mark visited; directed vs undirected

Also know: array of structs vs struct of arrays, bit flags and bit masks (central in embedded work), and strings as byte buffers with a terminator (C).

8. Core algorithm patterns

Learn these as patterns with trigger phrases. Most problems are a known pattern in disguise.

Trigger in the problem	Pattern	Core idea	Typical cost
sorted data, find something	binary search	halve the range each step	O(log n)
count, dedupe, find pairs, "seen before?"	hash map / set	remember what you have seen	O(n)
pair or range in sorted data; palindrome	two pointers	move inward from both ends	O(n)
best contiguous chunk; longest substring with a property	sliding window	extend the right edge, shrink the left	O(n)
range sums asked many times	prefix sums	precompute running totals	O(n) build, O(1) query
nested or matching; next greater element	stack	last in, first out	O(n)
fewest steps, shortest path (unweighted)	BFS	explore by levels with a queue	O(V+E)
explore everything, connected parts, cycles	DFS	recurse or use a stack; mark visited	O(V+E)
all combinations or permutations; constraints	backtracking	choose, recurse, undo the choice	exponential
top K; merge sorted streams; scheduling	heap	keep only the best K, or next-smallest	O(n log k)
a local best choice is always safe	greedy	sort, then take; prove it is safe	often O(n log n)
overlapping subproblems; min, max or count of ways	dynamic programming	define the state, the recurrence, the base case	polynomial

Also know: recursion and base cases, converting recursion to a loop with an explicit stack, sorting (insertion, merge, quick; in practice you call the library sort), and Big-O well enough to say why a solution is O(n) or O(n squared).

9. Failure modes and fixes

Symptom	Likely cause	Fix
Freeze at the blank file	Skipped the Input/Output sentence and the by-hand example	Write the sentence; work one case on paper
Code works but every change breaks something	Wrong data shape; parallel arrays; duplicated facts	Revisit nouns and collection; one record per thing; store raw facts only
One giant function	No top-down outline	Write the outline as comments; extract each step
Bugs only at edges	Edge cases not listed up front	Fill worksheet row 7 before coding; add each to a test
Same logic copied in several places	A function or type is missing	Name the repeated idea and extract it
Cannot test without hardware or user input	I/O mixed into logic	Pull logic into pure functions; keep I/O thin

Symptom	Likely cause	Fix
Pasting snippets that nearly fit	Design skipped; searching for solutions, not syntax	Design first; look up only syntax and library calls
Over-engineering	Building for needs you imagined	Smallest slice; add only what a real case needs
Understand each solution but cannot produce one	Reading, not generating	Close the reference; do the worksheet; then compare

10. The drill ladder

Work one language (C pairs with CS:APP and ESP32 work because you see memory directly; Rust teaches modeling well through enums, Option and Result, and its compiler forces the ownership questions). Do not switch mid-ladder. Finish each project before starting the next.

Level	Focus	Projects (50 to 150 lines each)	Done when
1	One type, one array	battery test log; paycheck budget calculator; temperature statistics (min, max, average)	Worksheet filled before code; every function called from main with known inputs; empty and full cases handled
2	Lookup, order, state	word frequency counter (hash map); bracket matcher (stack); ESP32 ring buffer for sensor readings; traffic-light state machine (enum)	You can say why each collection fits; you named the access pattern before choosing
3	Build the structures	dynamic array; linked list; hash map; binary heap; small BST	Each has tests; you can state its invariants and costs from memory
4	Compose	tokenizer plus calculator; grid or maze shortest path (BFS); key-value store with file persistence; command-line todo with priorities (heap)	Top-down outline first; pure logic separated from I/O; you could explain every type choice

- After level 2, add pattern problems (NeetCode or Blind 75 lists, sorted by pattern). For each: run the worksheet, name the pattern, solve it, then read other solutions and note what you would steal.
- Redo one finished project from a blank file two weeks later without looking at your old code. The difference between attempts measures the skill.
- Target volume: roughly 15 to 30 small finished programs. The early ones feel clumsy; that is expected.

11. Modeling drills (answer on paper)

For each scenario, fill worksheet rows 2 to 5 only: nouns with fields, enums, the collection and why, and verb signatures. Write types and signatures, no bodies. Bring your answers to chat and I will question them rather than hand you the answer.

#	Scenario	What to decide
1	AutoZone-style parts catalog: SKU, name, price, stock, bin location. Staff look parts up by SKU, and sometimes list everything in one bin.	Types; primary lookup structure; how bin listing is supported; behavior when a SKU is missing
2	ESP32 reads a sensor every 100 ms; only the last 50 readings matter; you need the average and the number of readings above a threshold.	Structure; behavior when full; what is stored per reading; pure vs hardware functions
3	Parking lot with 40 spaces: cars enter, leave, and the lot reports free spaces and who is parked where.	Types for space and car; lookup by plate vs by space number; invariants
4	Todo list where each item has a priority and a due date, and you always want the most urgent next.	Structure for "most urgent next"; what "urgent" compares; how edits work

#	Scenario	What to decide
5	Door lock: states locked, unlocked, alarm; events keypad correct, keypad wrong x3, timeout.	State enum; event enum; the transition function signature; what must be impossible
6	Playlist: songs with title and length, play in order, shuffle, skip, go back.	Collection; how "go back" works; the invariant on the current position

12. After-project review (answer every time)

- Which design decision caused the most trouble, and what would I model differently?
- Which pattern or structure did this problem belong to? Could I have named it before coding?
- Which edge case did I miss at the worksheet stage?
- Is any function doing more than one job? Any fact stored in two places?
- What did I look up, and was it syntax or design? (Design lookups mean the worksheet was incomplete.)
- How would an experienced programmer structure this? What do I take from their version?

13. Rust note (same loop, different tools)

The worksheet is identical. Rust moves several modeling decisions into the type system. Sketch of the battery log (types and signatures only; not compiled here):

```
enum Verdict { Good, Charge, Replace }

struct BatteryTest { id: u32, rated_cca: u32, measured_cca: u32, voltage: f64, verdict: Verdict }

struct TestLog { tests: Vec<BatteryTest> }

fn classify(rated_cca: u32, measured_cca: u32, voltage: f64) -> Verdict { todo!() }
fn avg_voltage(log: &TestLog) -> Option<f64> { todo!() }   /* None for an empty log: edge case in the type */
```

Note how `Option<f64>` encodes the empty-log edge case in the signature, so the caller cannot forget it. This is "make bad states impossible" built into the language.

14. Resources

Topic	Resource	Why
Decomposition and design	How to Design Programs (free, htdp.org)	The design recipe teaches this exact skill
	A Philosophy of Software Design, John Ousterhout	Cutting modules, managing complexity
	The Practice of Programming, Kernighan and Pike	Short, practical, C-flavored
	Programming Pearls, Jon Bentley	Thinking about the problem before the code
Data modeling	"Parse, don't validate" (Alexis King, blog)	Types that make bad states impossible
	"Make illegal states unrepresentable" (Yaron Minsky, idea and talks)	Same principle, in OCaml/Rust terms
Structures and algorithms	Algorithms, Jeff Erickson (free online)	Rigorous and readable
	Grokking Algorithms	Gentle first pass
	Algorithms, Sedgewick and Wayne	Standard reference with implementations

Topic	Resource	Why
	NeetCode roadmap	Practice sorted by pattern
Language	C: The C Programming Language (K&R); Beej's Guide to C	Core C and idioms
	Rust: The Rust Programming Language (the Book); Rust by Example; Rustlings	Core Rust
Practice	Exercism (C and Rust tracks); Advent of Code; Project Euler	Small problems, feedback

One-paragraph summary. Say input and output in one plain sentence. Nouns become types, access patterns choose collections, verbs become functions. Write types and signatures before logic. Run an example by hand, get the smallest slice working, and test each function as you go. Learn the eight standard structures and the dozen patterns so problems look familiar. Then repeat on many small projects, reviewing each one.